

RepoLens

AI-Powered Codebase Intelligence & Developer Onboarding Platform

Project continuity document — updated current state through React ↔ FastAPI communication, backend environment setup, UI design system, architecture direction, and future scope.

1. Project Vision

RepoLens is a full-stack platform that helps developers understand unfamiliar GitHub repositories. A user provides a repository URL, and the eventual system analyzes the repository structure, code, dependencies, and relationships between components. The AI then answers repository-specific questions, explains code, helps locate implementation details, and advises where and how a new feature should be changed.

Important development philosophy: build progressively for learning. Do not jump directly to LLM/RAG. First establish a reliable React → FastAPI → GitHub workflow, then add repository analysis, AI explanation, RAG, architecture intelligence, and change advice.

2. Product Goals

Goal	What RepoLens should eventually provide
Repository understanding	Project purpose, technologies, folders, files, dependencies and important components.
Code discovery	Find the exact file/function/block responsible for a feature such as login or signup.
Code explanation	Explain selected files or code using repository context.
Architecture intelligence	Show relationships among frontend, APIs, services, models and databases.
Change Advisor	Recommend files/functions to modify for a requested feature and estimate risk.
Onboarding	Generate a repository-specific developer onboarding guide.

3. Current Technology Stack

Layer	Current / Planned Technology
Frontend	React + Vite + JavaScript + CSS + React Router
Backend	Python + FastAPI + Uvicorn
Python environment	Project-local virtual environment (.venv)
Database	PostgreSQL + SQLAlchemy (planned)
Authentication	FastAPI + PostgreSQL + JWT (planned); GitHub OAuth later
GitHub	GitHub API / repository access (next backend milestone)
AI	LLM API, embeddings, vector database and RAG (planned)
Deployment	Potentially Vercel for frontend and Render/Railway for backend + PostgreSQL cloud

4. Current Project Structure

The project separates the frontend and backend so the React UI and FastAPI API can be developed independently. The backend environment and FastAPI server are now working.

```
RepoLens/
├── .env/                ← Python virtual environment
├── Frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Landing.jsx
│   │   │   ├── Landing.css
│   │   │   ├── Login.jsx
│   │   │   ├── Login.css
│   │   │   ├── Signup.jsx
│   │   │   ├── Signup.css
│   │   ├── App.jsx
│   │   ├── ...
│   ├── package.json
│   ├── ...
├── Backend/
│   ├── main.py
```

5. Frontend Pages Completed

- Login page:** RepoLens-themed UI with email/password fields, forgot-password link, sign-in button, account creation link and GitHub button. Authentication is currently UI-only.
- Signup page:** Matching UI with name, email, password, confirm-password fields, account creation button, login link and GitHub button. Password confirmation is currently handled in frontend JavaScript.
- Routing:** React Router connects / to Landing, /login to Login, and /signup to Signup.

6. Landing Page — Final Design Layout

Area	Purpose / Current state
Navbar	RepoLens brand on left; Login and Sign Up on right.
Hero	Short product statement explaining AI-powered codebase intelligence.
Repository input	GitHub URL field + Analyze Repository button; now connected to FastAPI.
Core capabilities	Three primary cards: Repository Overview, Find & Explain Code, Change Advisor.
How it works	Four simple stages: Connect → Analyze → Ask → Understand.
Footer	RepoLens identity, login/signup links and future GitHub link.

7. UI Design System

The visual identity is a dark developer-tool aesthetic: dark navy/charcoal surfaces, cool blue-to-teal gradients, subtle borders, soft glows, monospace labels, and clean technical typography.

Token	Value	Use
Background	#090C12	Main page background
Secondary background	#0D1119	Gradients and secondary areas
Surface	#11151F	Cards and elevated UI
Surface hover	#151B27	Interactive card hover

8. Gradient & Visual Rules

Primary accent gradient: **#7C9CFF** → **#4FD1C5**. It is used for primary CTA buttons, highlighted text, logo strokes and selected accents. Hero backgrounds use subtle radial gradients rather than strong flat colors. Cards use dark linear gradients and thin borders. Avoid introducing unrelated bright colors unless a semantic status (for example risk level) requires it.

Typography: Space Grotesk for display/headings, Inter for body/UI text, and JetBrains Mono for technical labels, metadata and code-oriented UI.

9. Important React Concepts Already Used

Concept	Purpose in RepoLens
Component	Landing.jsx, Login.jsx and Signup.jsx are reusable React components.
JSX	HTML-like syntax used inside JavaScript to describe the UI.
className	Connects JSX elements to CSS classes.
useState	Used in Login/Signup for form input and now used in Landing for the repository URL and backend response.
Link	React Router navigation between Landing, Login and Signup.
Routes / Route	Maps URLs such as /login and /signup to components.
onChange	Reads user input and updates React state.
Form / submit	Handles user form actions; repository Analyze currently uses a button click.
fetch()	Browser API used by React to send the GitHub URL to FastAPI and read the JSON response.

10. Backend Concepts Completed

Concept	Purpose in RepoLens
Python virtual environment	Keeps RepoLens backend packages isolated from other Python projects.
FastAPI	Framework used to create the backend application and API endpoints.
Uvicorn	ASGI server used to run the FastAPI application locally.
FastAPI app	Created with <code>app = FastAPI()</code> inside <code>Backend/main.py</code> .
GET /	Basic health endpoint confirming the RepoLens backend is running.
POST /analyze	Receives a <code>repo_url</code> and currently returns a confirmation JSON response.
CORS	Allows the React development server at <code>http://localhost:5173</code> to communicate with FastAPI.
JSON	Data format currently returned from the backend to React.

11. Current Status

Feature	Status
React + Vite setup	Completed
Login UI	Completed
Signup UI	Completed
Login ↔ Signup routing	Completed
Landing page UI	Completed
Landing → Login/Signup navigation	Completed
Python virtual environment	Completed
FastAPI setup	Completed
Uvicorn setup	Completed
CORS configuration	Completed
React → FastAPI communication	Completed
GitHub URL → /analyze	Working
Real authentication	Not started
GitHub repository import	Next step
Repository analysis	Future
AI/RAG	Future

12. What Has Actually Been Tested

The current full-stack connection has been tested successfully. The user enters a GitHub URL on the Landing page, React stores it in state, the Analyze button calls the FastAPI endpoint with `fetch()`, FastAPI receives the URL, and React displays the returned message.

```
Browser / React
http://localhost:5173
  ┆
  ┆ fetch() → POST /analyze
  ▼
FastAPI / Uvicorn
http://127.0.0.1:8000
  ┆
  ┆ JSON response
  ▼
React displays: "Repository URL received"
```

The backend terminal has also returned **200 OK** for the POST /analyze request, confirming that the request reached FastAPI successfully.

13. Immediate Next Milestone

Do not start AI yet. The next milestone is to make the /analyze endpoint useful by connecting FastAPI to the GitHub API and retrieving basic repository information.

Target flow:

```
Landing page → User enters GitHub URL
→ Analyze button → React sends URL
→ FastAPI receives URL
→ FastAPI calls GitHub API
→ FastAPI retrieves repository metadata
→ FastAPI returns JSON
→ React displays repository information
```

The first GitHub milestone should retrieve repository name, owner, description, languages, folders and files. Only after that works should the project move toward repository analysis and AI.

14. Full Future Roadmap

Phase	Main objective
Phase 1	Frontend foundation: React, Vite, routing, Login, Signup, Landing.
Phase 2	React ↔ FastAPI communication and GitHub repository import. Current phase; basic React ↔ FastAPI connection completed.
Phase 3	Repository structure analysis: files, folders, languages, dependencies, README and configuration.
Phase 4	LLM-powered project overview and file/folder explanations.
Phase 5	Code chunking + embeddings + vector search + RAG for repository-aware Q&A.;
Phase 6	Architecture intelligence and visualization of frontend/API/service/database relationships.
Phase 7	Change Advisor: relevant files, functions, dependencies, likely changes and risk.
Phase 8	Repository-specific developer onboarding guide.
Phase 9	Advanced features: risky files, contribution suggestions, dependency analysis, code quality, test coverage, PR analysis and repository health score.

15. Final Product Direction

The finished RepoLens portfolio project should demonstrate both software engineering and applied AI. It should not look like a generic chatbot or CRUD application. The central differentiator is that answers and recommendations are grounded in the actual repository being analyzed.

Final CV description: RepoLens — AI-Powered Codebase Intelligence & Developer Onboarding Platform. A full-stack platform that analyzes GitHub repositories, explains codebase architecture, enables repository-aware AI Q&A, generates developer onboarding guides, and recommends code changes for new features.

16. Current Build Instructions for Future AI Chats

Treat this document as the project source of truth. Preserve the existing RepoLens name, dark UI system, #7C9CFF → #4FD1C5 accent gradient, current page structure, React Router paths, and progressive development philosophy.

Current starting point: Frontend UI is already built. Backend Python environment, FastAPI and Uvicorn are already configured. CORS is configured for the React development server. The Landing page already sends the GitHub URL to **POST /analyze** using `native fetch()`, and the backend successfully returns JSON that React displays.

Do not redo completed setup or rewrite completed pages unnecessarily. The next implementation should build incrementally from the working `/analyze` endpoint and connect it to GitHub repository metadata retrieval. Explain unfamiliar concepts before introducing them.

Current working local servers: React development server at `http://localhost:5173` and FastAPI/Uvicorn server at `http://127.0.0.1:8000`.